Week1_예습과제

≡ 주차1	1주차

파이썬 기반의 머신러닝과 생태계 이해

1. 머신러닝의 개념

머신러닝 : 애플리케이션 수정 없이도 **데이터를 기반으로 패턴 학습+결과 예측**하는 알고리즘 기법

- 복잡한 조건/규칙들이 복잡+다양하게 결합+시시각각 변화 → 일정한 패턴 파악이 어려울 경우 👍
- 데이터 기반 + 다양한 수학기법 적용 → 신뢰도 ↑

1) 머신러닝의 분류

- 지도학습
  - 분류
  - 회귀
  - 추천 시스템
  - 시각/음성 감지/인지
  - 텍스트 분석, NLP
- 비지도학습
  - 클러스터링
  - 차원 축소
  - 강화학

2) 데이터 전쟁

: 데이터 vs 머신러닝 알고리즘

- Garbage In, Garbage out
- 데이터 이해도&최적의 데이터 준비 능력 > 최적의 머신러닝 알고리즘&모델 파라미터 구축 능력

3) 파이썬과 R 기반의 머신러닝 비교

- R : 통계 전용 프로그램 언어
- 파이썬 : 광범위하게 사용되는 개발 전문 프로그램 언어
  - 직관적인 문법, 객체지향, 함수형 프로그래밍, 유연한 프로그래밍 아키텍처, 다양한 라이브러리, 딥러닝 프레임워크 지원 ↑

⇒ 개발자 or 머신러닝 시작할 시 파이썬 권장

2. 파이썬 머신러닝 생태계를 구성하는 주요 패키지

- 머신러닝 패키지
  - 사이킷런(Scikit-Learn)
- 행렬/선형대수/통계 패키지
  - 넘파이(Numpy) : 파이썬의 대표적인 행렬과 선형대수 다루는 패키지
  - 사이파이(SciPy) : 자연과학과 통계를 위한 다양한 패키지
- 데이터 핸들링
  - 판다스(Pandas) : 2차원 데이터 처리에 특화됨
- 시각화
  - 맷플롯립(Matplotlib) : 대표적 but 세분화 ↑ &디자인 투박 → 효율↓ & 불편한 점이 있음
  - 시본(Seaborn) : 맷플롯립을 보완하여 함축적으로 만듦 but 맷플롯립에 대한 이해 필요

- 주피터 노트북(Jupyter notebook) : 대표적인 아이파이썬(IPython/대화형 파이썬 툴) 지원 툴

3. 넘파이(Numpy)

- 선형대수 기반의 프로그램을 쉽게 만들 수 있도록 지원
- 루프 없이 대량 데이터의 배열 연산 가능
- C/C++과 같은 저수준 언어 기반의 호환 API 제공
- 다양한 데이터 핸들링 기능 제공

넘파이 모듈 임포트 : `import numpy as np`

1) 넘파이 ndarray 개요

〈 넘파이 ndarray 배열의 차원들 〉

ndarray : 넘파이의 기반 데이터 타입

`array( )` 함수	리스트 등을 인자로 받아 ndarray로 변환	array1 = np.array([l, 2, 3])
`shape` 변수	ndarray의 크기(열,행)을 튜플 형태로 가짐 → 차원 알 수 있음	print(array 1 .shape) >>(3,)

💡 np.array ([[1,2,3]]) vs np.array([1,2,3])

	차원	array3.shape
np.array ([[1,2,3]])	2	(3,)
np.array ([1,2,3])	1	(3,1)

2) ndarray의 데이터 타입

- 숫자 값, 문자열 값, 불 값 등 모두 가능
- ndarray 내의 데이터 타입은 그 연산의 특성상 **같은 데이터 타입**만 가능

  ⇒ 만약 다른 유형이 섞임 → 데이터 크기가 더 큰 데이터 타입으로 형 변환을 일괄 적용

  e.g.) string형&int형→int형이 string형으로 변환

`dtype` 속성	ndarray 내 데이터 타입 확인 가능	`array1.dtype`
`astype( )` 메소드	인자로 원하는 타입을 문자열로 지정 하면 ndarry 내 데이터값의 타입 변환 → 메모리 절약에 이용	`array_int=np.array([1,2,3])` `array_float=array_int.astype('float64')`

3) ndarray를 편하게 생성하기 - arange, zeros, ones

`arang( )` 함수	`range( )` 함수와 유사 → (시작,끝,건너뛸 수)	`equence_array=np.arange(10)`
`zeros( )` 함수	인자로 튜플 형태의 shape 값을 입력 dtype 정하지 않으면 float64	`zero_array=np.zeros((3,2),dtype='int32')`
`ones( )` 함수	인자로 튜플 형태의 shape 값을 입력 dtype 정하지 않으면 float64	`one_array=np.ones((3,2))`

4) ndarray의 차원과 크기를 변경하는 reshape()

`resshape( )` 메소드	ndarray를 특정 차원 및 크기로 변환 • 원하는 크기를 인자로 • 지정된 사이즈로 변경 불가→오류 • **-1 인자** : 나머지 인자를 기준으로 자동 변환 - but 이 경우도 변경 불가하면 오류 - (-1,1) 형태로 자주 사용 : 여러 개의 로우+반드시 1개의 컬럼을 가진 **2차원 ndarray**	`array2=array1.reshape(2,5)`
`tolist( )` 메소드	ndarray를 리스트로 변환 → 리스트가 출력 시 시각적 이해 ↑인 경우 사용	`array3d.tolist()`

5) 넘파이의 ndarray의 데이터 세트 선택하기 - 인덱싱(Indexing)

- 인덱싱
  - 특정한 데이터만 추출 : 원하는 위치의 인덱스 값을 지정하면 해당 위치의 데이터 반환
  - 슬라이싱(Slicing) : 연속된 인덱스상의 ndarray를 추출 / ':' 기호 사용
  - 팬시 엔덱싱(Fancy Indexing) : **일정한 인덱싱 집합을 리스트 또는 ndarray 형태로 지정**해 해당 위치에 있는 데이터의 ndarray를 반환
  - 불린 인덱싱(Boolean Indexing) : True/False 값 인덱싱 집합을 기반으로 **True에 해당하는 인덱스 위치**에 있는 데이터의 ndarray를 반환

(1) 단일 값 추출

1차원	array[위치 인덱스]	`array1[-1]`
다차원	array[axis 0의 위치 인덱스, axis 1의 위치 인덱스, ... ,axis N의 위치 인덱스]	`array2d[1,0]`

- 추출된 데이터는 ndarray 타입이 아니고 ndarray 내의 데이터 값
- 단일 인덱스로 추출 뿐만 아니라 테이터 수정도 가능
- 위치 인덱스는 0부터 시작함을 주의

(2) 슬라이싱

- ':' 기호를 이용해 연속한 데이터를 슬라이싱해서 추출 가능

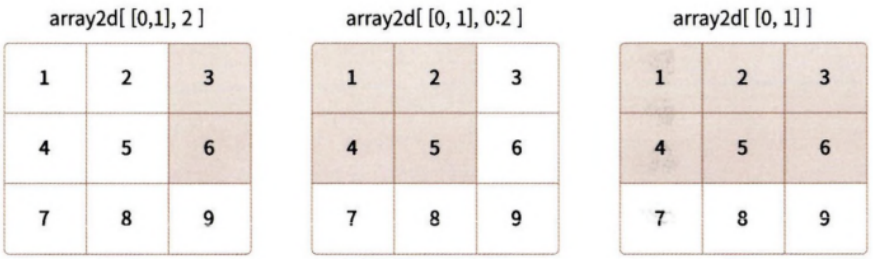
  → ':' 기호 앞/뒤에 시작/종료 인덱스를 생략하면 자동으로 맨 마지막 인덱스로 간주
- **추출된 데이터 세트는 모두 ndarray 타입**

1차원	array[시작 인덱스 : 종료 인덱스]	`array1[:3]`
다차원	array[시작 인덱스 : 종료 인덱스, ... , 시작 인덱스 : 종료 인덱스] • 한쪽에만 슬라이싱 적용 + 다른 쪽 축에는 단일 값 인덱스 적용 가능 • n차원 ndarray에서 뒤에 오는 인덱스를 없애면 n-1차원 ndarray 반환 - e.g.) array2d[0] → 첫번째 로우 ndarray를 1차원으로 반환	`array2d[1,0]` `array2d[:2,0]`

(3) 팬시 인덱싱

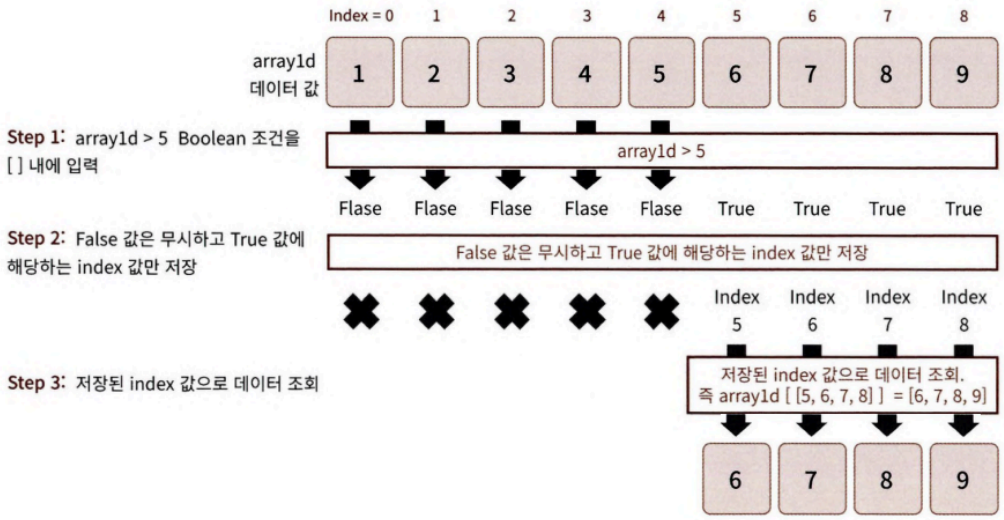
- 리스트나 ndarray로 인덱스 집합을 지정하면 해당 위치의 인덱스에 해당하는 ndarray를 반환하는 인덱싱 방식

팬시 인덱싱 + 팬시 인덱싱	리스트의 n번째 있는 수끼리 좌표를 만듦	`arr_2d[[0, 2], [0, 1]]` >>((0,0),(2,1)) 인덱싱 적용
팬시 인덱싱 + 단일값 인덱싱	로우 축에 팬시 인덱싱 [0,1] 적용 + 칼럼 축에 단일 값 인덱싱 2를 적용 → 0행 1행에서 2열 값 반환	`array2d[[0,1],2]`
팬시 인덱싱 + 슬라이싱	0행 1행에서 0에서 1열까지의 값 반환	`array2d[[0,1],0:2]`
팬시 인덱싱	0행 1행 전체를 가져옴	`array2d[[0,1]]`



(4) 불린 인덱싱

- 조건 필터링과 검색을 동시에 할 수 있음
- 원리



> 💡 일반 인덱스로도 필터링 가능
> `array1d=np.arange(start=1,stop=10)`
> `indexes=np.array([5,6,7,8])`
> `array4=array1d[indexes]`
>
> `print('일반 인덱스로 필터링한 결과 :',array4)`
>
> >> 일반 인덱스로 필터링한 결과 : [6 7 8 9]

6) 행렬의 정렬 - sort( )와 argsort( )

(1) 행렬 정렬

- `np.sort( )` vs `ndarray.sort( )`

		원행렬	반환값	예시 코드
`np.sort( )`	넘파이에서 호출	그대로 유지	원 행렬의 정렬된 행렬	`sort_array1=np.sort(org_array)`
`ndarray.sort( )`	행렬 자체에서 호출	원 행렬 자체를 정렬	None	`sort_array2=org_array.sort()`

- 오름차순 정렬이 기본

  → 내림차순 정렬 : `np.sort( )[ : : -1]`
- 다차원 행렬 정렬 : axis 축 값 설정
  - 2차원 로우 방향(axis 0) 정렬 : `np.sort(array2d,axis=0)`
  - 2차원 칼럼 방향(axis 1) 정렬 : `np.sort(array2d,axis=1)`

(2) 정렬된 행렬의 인덱스를 반환하기

- `np.argsort( )`는 **정렬 행렬의 원본 행렬 인덱스**를 **ndarray 형**으로 반환

  → 내림차순 정렬 시의 원본 행렬 인덱스 구하기 : `np.argsort( )[ : : -1]`
- 넘파이의 ndarray는 메타 데이터를 가질 수 없음 → 별도로 ndarray를 각각 가져야 함 (같은 ndarray에는 같은 타입의 데이터만 넣을 수 있음) → array1의 바뀐 정렬을 array2에 적용할 때 많이 사용

7) 선형대수 연산 - 행렬 내적과 전치 행렬 구하기

(1) 행렬 내적(행렬 곱)

- `np.dot(A,B)` 이용
- '행렬A의 열 개수 = 행렬B의 행 개수'여야 함

(2) 전치 행렬

- 원 행렬에서 행과 열의 위치를 교환한 원소로 구성한 행렬
- A의 전치행렬은 A^T로 표
- `np.transpose(A)` 이용

---

4. 데이터 핸들링 - 판다스

- 행과 열로 이루어진 2차원 데이터를 효율적으로 가공/처리 가
- DataFrame : 판다스 핵심 개체 / 2차원 데이터를 담는 데이터 구조체 / 칼럼이 여러 개인 데이터 구조체
- Index : 개별 데이터를 고유하게 식별하는 Key 값 / 칼럼명 표시X인 맨 왼쪽 열

  → Series와 DataFrame 모두 Index를 key 값으로 가짐

  → Series, DataFrame가 만들어진 후에도 변경 가능

  → 숫자형, 문자열 가능
  → 고유성이 보장되어야 함
- Series : 칼럼이 하나 뿐인 데이터 구조체 / Index와 단 하나의 칼럼으로 구성된 데이터 세

1) 판다스 시작 - 파일을 DataFrame으로 로딩, 기본 API

- 다양한 포맷으로 된 파일을 DataFrame으로 로딩할 수 있는 API

`read_csv( )`	기본적으로 콤마(',')로 구분된 csv 파일 포맷 변환 **sep 인자로 구분 문자 지정 가능** **filepath 인자** : 데이터 파일의 경로를 포함한 파일명 입력
`read_table( )`	탭('\t')로 구분된 csv 파일 포맷 변환
`read_fwf( )`	FIxed Width, 즉 고정 길이 기바나의 칼럼 포맷을 DataFrame으로 로딩

`DataFrame.head( )` `Series.head( )`	DateFrame의 맨 앞에 있는 N개의 로우 반환 Series의 맨 앞에 있는 N개의 로우 반환
`DataFrame.shape`	DateFrame 객체의 shape 변수 이용 → 행,열 크기를 튜플 형태로 반환
`DataFrame.info( )`	• 총 데이터 건수 • 데이터 타입 • Null건수 를 알 수 있음
`DataFrame.describe( )`	칼럼별 숫자형 데이터값의 • n-percentile 분포도 • 평균값 • 최댓값 • 최솟값 와 Not null(count)를 알 수 있 → objectタイ입의 칼럼은 출력에서 자동으로 제외 **개략적인 수준의 데이터 분포도를 확인 가능** 숫자형 카테고리 칼럼인지 판단 가능 (e.g. 남=1,여=2)
`DataFrame['칼럼명']`	**Series 형태**로 특정 칼럼 데이터 세트 반환
`Series.value_counts( )`	• 칼럼 값별 데이터 건수를 Series로 반환 → Index에서 고유 칼럼 값을 식별자로 사용 가능 • dropna 인자 : Null 값 포함하여 개별 데이터 값의 건수를 계산할지 정함 → 기본 값은 True (무시) / False이면 NaN(Not a Number)도 나옴 • Dateframe 객체에서도 호출 가능 **but** Series 객체에서 호출하는 것이 칼럼별 데이터 값 분포도의 명시적 파악이 쉬움

2) DataFrame과 리스트, 딕셔너리, 넘파이 ndarray 상호 변환

- DataFrame ↔ 리스트, 딕셔너리, 넘파이 ndarray 변환 가능

(1) 넘파이 ndarray, 리스트, 딕셔너리를 DateFrame으로 변환하기

- 생성인자 data : 리스트, 딕셔너리, 넘파이 ndarray (2차원 이하)
- 생성 인자 columns : 칼럼명 리스트 (지정X → 자동으로 할당)
  - 리스트, ndarray를 변환할 때 : 칼럼명은 1차원일 경우 1개, 2차원일 경우 열의 개수 만큼 필요
  - 딕셔너리를 변환할 때 : key 값은 칼럼명, 각 value는 각 칼럼 데이터로 매핑됨

(2) DateFrame을 넘파이 ndarray, 리스트, 딕셔너리로 변환하기

- DataFrame 객체의 values를 이용해 ndarray로 변환 가능
  - `DataFrame.values`

ndarray로 변환	DataFrame 객체의 values를 이용해 ndarray로 변환 가능	`DataFrame.values`
리스트로 변환	values로 얻은 ndarray에 tolsit( )를 호	`DataFrame.values.tolist()`
딕셔너리로 변환	DataFrame 객체의 `to_dict( )` 메서드 호출 → 인자로 'list' 입력하면 딕셔너리의 값이 리스트형으로 반환	`DataFrame.to_dict('list')`

3) DataFrame의 칼럼 데이터 세트 생성과 수정

- [ ] 연산자로 DataFrame의 칼럼 데이터 세트 생성 및 수정 가능
  - Series에 상숫값 할당 → 모든 데이터 세트에 일괄 적용
  - 칼럼끼리 더한 새로운 칼럼 생성 / 기존 칼럼에 수식을 더해 수 등이 가능

    `titanic_df['Age_by_10'] = titanic_df['Age'] * 10`

    `titanic_df['Age_by_10'] = titanic_df['Age_by_10'] + 100`

4) DataFrame 데이터 삭제

- `drop( )` 메소드 이용
  - 원형 : `DataFrame.drop( labels=None , axis=0 , index=None, columns=None, level=None, inplace=False , errors='raise')`
    - axis=1로 설정 = 지정된 칼럼 드롭 → labels에 원하는 칼럼명 입력

      → 기존 칼럼 값을 가공해 새로운 칼럼을 만들고 삭제할 때 사용
    - axis=0으로 설정 = 지정된 인덱스 드롭 → labels에 원하는 인덱스 입력

      → 이상치 데이터 삭제에 사용

axis=1로 설정 = 지정된 칼럼 드롭 (기존 칼럼 값을 가공해 새로운 칼럼을 만들고 삭제할 때 사용)	labels에 원하는 칼럼명 입력 • 리스트로 묶어 여러 열 삭제 가능	`titanic_drop_df = titanic_df.drop('Age_0', axis=1)`
axis=0으로 설정 = 지정된 인덱스 드롭 (이상치 데이터 삭제에 사용)	labels에 원하는 인덱스 입력 • 리스트로 묶어 여러 데이 삭제 가능	`titanic_drop_df = titanic_df.drop(5, axis=0)` (= `drop(index=5)`)
inplace 인자	• False(기본값) : 자기 자신의 데이터는 삭제X + 삭제된 결과를 반환 • True : 자기 자신의 데이터에서 삭제 + **반환값 X (→ 자신의 DF 객체에 다시 반환값을 할당하면 None이 됨을 주의)**	`drop_result = titanic_df.drop(['Age_0', 'Age_by_10'], axis=1, inplace=True)`

5) Index 객체

`DataFrame.index`
`Series.index`
	Index 객체만 추출 가능
	→ 여기에 `.values` 를 붙이면 ndarray가 됨
		`print(indexes[:5].values)`
		>> [0 1 2 3 4]
		`print(indexes[6])`
		>> 6

- ndarray처럼 슬라이싱 및 단일값 반환 가능

- Series 객체는 Index 객체를 포함 but Series 객체에 연산 함수(`.max( )`, `.sum( )`...)를 적용할 때 Index는 연산에서 제외됨
- 오직 식별용
- 한 번 만들어진 Index 객체 함부로 변경 불가

`DataFrame.reset_index( )` `Series.reset_index( )`	새롭게 인덱스를 연속 숫자 형으로 할당 기존 인덱스는 'index'라는 새로운 칼럼명으로 추가 Series에 적용하면 DataFrame이 반환 파라미터 `drop=True`로 설정하면 기존 인덱스는 새로운 칼럼으로 추가되지 않고 삭제됨 -> Series로 유지됨	`new_value_counts = value_counts.reset_index(inplace=False)`

6) 데이터 셀렉션 및 필터링

- 넘파이와 혼동하지 않도록 주의해야 함

(1) DataFrame의 [ ] 연산자

- DataFrame 바로 뒤의 [ ] 연산자는 넘파이의 [ ]나 SEries의 [ ]와 다름을 주의

넘파이	행의 위치, 열의 위치, 슬라이싱 범위 등을 지정해 데이터를 가져올 수 있음	

판다스의 DataFrame	• 칼럼명 문자
	• 칼럼명의 리스트 객체
	• 인덱스로 변환 가능한 표현식(행 슬라이싱, 불린 인덱싱)을 연산자 안에 넣을 수 있음
	→ 칼럼 지정 연산자
	*) 숫자 값 입력 시 오류 발생
		`titanic_df['Pclass']`
		`titanic_df[['Survived', 'Pclass']]`
		`titanic_df[0:2]`
		`titanic_df[titanic_df['Pclass'] == 3]`

(2) DataFrame iloc[ ] 연산자

- **위치(Location) 기반 인덱싱** 방식으로 동작

  → 행과 열 위치를 0을 출발점으로 하는 세로축, 가로축 좌표 정숫값으로 지정하는 방식
- 슬라이싱과 팬시 인덱싱은 제공하나 **불린 인덱싱은 제공 X**

iloc[ ] 연산 유형	설명 및 반환 값
data_df.iloc[1,0]	두번째 행의 첫번째 열 위치에 있는 단일 값 반환 반환 값: 'Eunkyung'
data_df.iloc[2,1]	세번째 행의 두번째 열 위치에 있는 단일 값 반환 반환 값: '2015'
data_df.iloc[0:2, [0,1]]	0:2 슬라이싱 범위의 첫번째에서 두번째 행과 첫번째, 두번째 열에 해당하는 DataFrame 반환 반환 값: Name Year one Chulmin 2011 two Eunkyung 2016
data_df.iloc[0:2, 0:3]	0:2 슬라이싱 범위의 첫번째에서 두번째 행의 0:3 슬라이싱 범위의 첫번째부터 세번째 열 범위에 해당하는 DataFrame 반환 반환 값: Name Year Gender one Chulmin 2011 Male two Eunkyung 2016 Female
data_df.iloc[:]	전체 DataFrame 반환 Name Year Gender one Chulmin 2011 Male two Eunkyung 2016 Female three Jinwoong 2015 Male four Soobeom 2015 Male
data_df.iloc[:, :]	전체 DataFrame 반환 Name Year Gender one Chulmin 2011 Male two Eunkyung 2016 Female three Jinwoong 2015 Male four Soobeom 2015 Male

(3) DataFrame loc[ ] 연산자

- **명칭(Label) 기반 인덱싱** 방식으로 동작

  → 데이터 프레임의 인덱스 값으로 행 위치를, 칼럼의 명칭으로 열 위치를 지정하는 방식

  → loc[인덱스값, 칼럼명]

- 슬라이싱 기호 **':'** 사용 시 시작 값에서 **종료 값까지 뽑아냄** (→ 종료-1까지 뽑아내지 않음)

loc[ ] 연산 유형	설명 및 반환 값
data_df.loc['three', 'Name']	인덱스 값 three인 행의 Name 칼럼의 단일값 반환 반환 값: 'Jinwoong'
data_df.loc['one':'two', ['Name', 'Year']]	인덱스 값 one부터 two까지 행의 Name과 Year 칼럼에 해당하는 DataFrame 반환 반환 값: Name Year one Chulmin 2011 two Eunkyung 2016
data_df.loc['one':'three', 'Name':'Gender']	인덱스 값 one부터 three까지 행의 Name부터 Gender 칼럼까지의 DataFrame 반환 반환 값: Name Year Gender one Chulmin 2011 Male two Eunkyung 2016 Female three Jinwoong 2015 Male
data_df.loc[:]	모든 데이터 값: Name Year Gender one Chulmin 2011 Male two Eunkyung 2016 Female three Jinwoong 2015 Male four Soobeom 2015 Male
data_df.loc[data_df.Year >= 2014]	iloc[ ]와 다르게 loc[ ]는 불린 인덱싱이 가능. Year 칼럼의 값이 2014 이상인 모든 데이터를 불린 인덱싱으로 추출 Name Year Gender two Eunkyung 2016 Female three Jinwoong 2015 Male four Soobeom 2015 Male

(4) [ ], iloc[ ], loc[ ]의 특징과 주의할 점

- 개별 또는 여러 칼럼 값 전체를 추출할 때 → iloc[ ]나 loc[ ]를 사용하지 않고 DataFrame['칼럼명']만으로 충분
- 행과 열을 함께 사용항 데이터를 추출할 때 → iloc[ ]나 loc[ ]를 사용

(5) 불린 인덱싱

- [ ], loc[ ]에서 공통으로 지원(정수형 값 사용)
- DataFrame 객체 반환 → 불린 인덱싱 적용한 DataFrame에 칼럼명만 별로로 추출 가능
  - `titanic_df[titanic_df['Age'] > 60][['Name', 'Age']]`
  - `titanic_df.loc[titanic_df['Age'] > 60, ['Name', 'Age']]`
- 여러 개의 복합 조건 결합해 적용 가능
  - and 조건일 때는 &
  - or 조건일 때는 |

- Not 조건일 때는 ~
- `titanic_df[(titanic_df['Age'] > 60) &(titanic_df['Pclass'] == 1) & (titanic_df['Sex'] == 'female')]`

- [ ], ~~ix[ ]~~, loc[ ]에서 지원 됨

7) 정렬, Aggregation 함수, GroupBy 적용

(1) DataFrame, Series의 정렬 - sort_values()

- 주요 입력 파라미터
  - by : 특정 칼럼을 입력 → 해당 칼럼으로 정렬 수행 / 여러 칼럼 정렬 → 리스트 형식으로 입력
  - ascending :
    - ascending=True : 오름차순 정렬 (기본)
    - ascending=False : 내림차순 정렬
  - inplace :
    - inplace=False : sort_values를 호출한 DataFrame은 그대로 유지+정렬된 DataFrame 결과를 반환 (기본)
    - inplace=True : 호출한 DataFrame의 정렬 결과를 그대로 적용

(2) Aggregation 함수 적용

- e.g.) min( ), max( ),sum( ), count( )
- DataFrame에서 바로 aggregation을 호출할 경우 모든 칼럼에 해당 aggregation을 적용

  → 특정 칼럼에만 적용하고 싶을 때 : DataFrame에 대상 칼럼들만 추출해 aggregation을 적용
- count( )는 Null 값을 반영하지 않은 결과를 반환

(3) groupby( ) 적용

- groupby : 대상 칼럼이 같은 것끼리 묶음
- groupby( ) 사용 시 입력 파라미터 by에 칼럼 입력 → 대상 칼럼으로 groupby 됨
- DataFrame에 groupby( )를 호출 → DataFrameGroupBy 형태로 반환
- 보통 groupby( )로 묶고 aggregation으로 계산함

  → 특정 칼럼을 기준으로 행들 묶고 → 묶인 그룹 안에서 aggregation을 수행
  - `titanic_groupby = titanic_df.groupby('Pclass').count()`
- DataFrame의 groupby( )에 특정 칼럼만 aggregation 함수를 적용하고 싶을 때

  → grouby( )로 반환된 DataFrameGroupBy 객체에 해당 칼럼을 필터링한 뒤 aggregation 함수를 적용
- 여러 aggregation 함수를 적용하고 싶을 때

  → 여러 개의 aggregation 함수명을 DataFrameGroupBy 객체의 `.agg( )` 내에 인자로 입력해서 사용
  - `titanic_df.groupby('Pclass')['Age'].agg([max, min])`

- 함수명을 문자열로 입력하는 것을 권장 e.g.) `'max'`
- **여러 개의 칼럼에 서로 다른 aggregation함수**를 groupby에 호출

  → agg() 내에 입력값으로 **딕셔너리 형태**로 aggregation이 적용될 칼럼들과 aggregation 함수를 입력

  - `agg_format = {'Age': 'max', 'SibSp': 'sum', 'Fare': 'mean'}`
    `titanic_df.groupby('Pclass').agg(agg_format)`

8) 결손 데이터 처리하기

- 결손 데이터 = Missing Data = NULL인 경우 = 넘파이의 NaN
- NaN 값은 평균, 총합 등의 함수 연산 시 제외 됨
- 기본적으로 머신러닝 알고리즘은 NaN 값을 처리X → 다른 값으로 대체해야 함

(1) isna( )로 결손 데이터 여부 확인

- DataFrame에 isna( )를 수행 → 모든 칼럼의 값이 NaN인지 아닌지 True나 False로 알려줌
  → True는 1, False는 0으로 변환됨 → 결손 데이터의 개수 구하기 가능
- `titanic_df.isna().sum()`

(2) fillna( )로 결손 데이터 대체하기

- `DataFrame['칼럼명'].fillna(대체할 값)`
- 주의할 점
  - `filna( )`를 이용해 반환 값을 다시 받기 or `inplace=True` 파라미터를 `fillna( )`에 추가하기

    → 안 그러면 변경 X

    - `DataFrame['칼럼명']=DataFrame['칼럼명'].fillna(대체할 값)`
      or `DataFrame['칼럼명'].fillna(대체할 값,inplace=True)`

9) apply lambda 식으로 데이터 가공

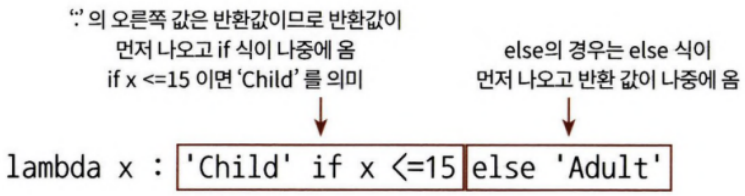
(0) lambda 함수

- 기본형 : `lambda 입력인자 : 반환될 입력 인자의 계산식`

  → 이 식을 함수명으로 받아서 사용 가능
- 여러 개의 값을 입력 인자로 사용할 경우 : `map( )` 함수 결합
  - `map(함수, 반복가능한데이터)` → map 객체 반환

(1) apply에 lambda 식을 적용해 데이터 가공

- `titanic_df['Name_len'] = titanic_df['Name'].apply(lambda x: len(x))`
- if else 절 :



- `lambda 인자 : 반환값 if 조건문 else 반환값`
- 주의할 점 : if 절의 식과 반환값의 위치 주의
- `else if` 는 지원X

  → else 절을 ( )로 내포해 ( ) 내에서 다시 if else를 적용해 사용

  - `titanic_df['Age'].apply(lambda x: 'Child' if x <= 15 else ('Adult' if x <= 60 else 'Elderly') )`
  - else if가 많이 나와야하는 경우 or switch case 문의 경우 아예 별도의 함수를 만드는 것을 추천

    → `DataFrame['칼럼명'].apply(lambda x: 함수명(x))`